What is a database, and why should you care?

Learning the idea with one month of data from a boiler house

YOU NEED

No experience with databases

TOOLS

A web browser. That is all.

BY THE END

You will answer a real question

TODAY

How the session runs

00:00	Why not just use a spreadsheet?	20 min
00:20	Tables, rows, columns and keys	25 min
00:45	Your first queries	15 min
01:00	**EXERCISE 1 — looking at the data**	25 min
01:25	Break	10 min
01:35	Summarising with GROUP BY	15 min
01:50	**EXERCISE 2 — summarising**	25 min
02:15	Joining two tables	20 min
02:35	**EXERCISE 3 — joining, and the answer**	30 min
03:05	What you learned, and where next	10 min

Half of today is you at the keyboard. Nothing is assessed.

THE PROBLEM

You already know why this is painful

One sheet, one row per reading. It starts out fine.

date	boiler	rating	operator	shift	steam_t	gas_gj
01/04	Boiler 2	35 t/h	Naree Wong	A	651	2050
01/04	Boiler 2	35 t/h	naree wong	A	648	2044

1 **The rating is typed on every single row**

6,000 rows later, someone re-rates the boiler to 38 t/h. Now you have to find and change all 6,000.

2 **The same person, spelled three ways**

“Naree Wong”, “naree wong”, “N. Wong”. Any count by operator is now wrong, and nothing warns you.

3 **Delete the last row for Boiler 2**

You have just deleted the only record that Boiler 2 is rated 35 t/h. The equipment data disappeared with the reading.

4 **Nothing stops you typing “high”**

A spreadsheet will happily accept text in a temperature column. So will the average you calculate from it.

THE IDEA

A database is just tables — with rules

A table is a list of one kind of thing. That is the whole definition.

This is the boilers table. One row = one boiler.

boiler_id	code	name	rating_tph	fuel
1	B-01	Boiler 1	35.0	Natural gas
2	B-02	Boiler 2	35.0	Natural gas
3	B-03	Boiler 3	30.0	Natural gas

Three rows, because there are three boilers. Five columns, because there are five things we want to know about each one.

Ask yourself

What is one row?

If you cannot answer that in four words, the table is trying to be two tables.

“One boiler.” “One reading.” “One day on one boiler.” Those are good tables.

The rules are what make it a database rather than a sheet: this column holds numbers and only numbers, this row must be unique, and this value must exist over in that other table. The computer enforces them, so you cannot quietly break your own data.

THE IDEA

Rows, columns, and one very important column

boiler_id	code	name	rating_tph	fuel
1	B-01	Boiler 1	35.0	Natural gas
2	B-02	Boiler 2	35.0	Natural gas
3	B-03	Boiler 3	30.0	Natural gas

▲

boiler_id is the key. Every row has one, no two rows share one, and it never changes.

A row is one boiler

Read across a row and you get everything the database knows about that boiler.

A column is one fact

Read down a column and you get the same fact for every boiler. One column, one meaning, one kind of value.

Rows have no order

The database does not remember which row is “first”. If you want an order, you ask for one. This surprises spreadsheet users more than anything else.

The key is the row's name

Everywhere else in the database, when we want to talk about Boiler 2, we just write 2.

THE IDEA

Why not keep everything in one big table?

Because the same fact ends up written down hundreds of times.

One big table

date	boiler	rating	steam_t	gas_gj
01/04	Boiler 2	35.0	651	2050
02/04	Boiler 2	35.0	648	2044
03/04	Boiler 2	35.0	655	2061
…	…	35.0	…	…

“35.0” appears on every row for the rest of time. Change the rating and you must change every copy — or the data quietly disagrees with itself.

Two tables

boiler_id	code	rating_tph
2	B-02	35.0

date	boiler_id	steam_t	gas_gj
01/04	2	651	2050
02/04	2	648	2044
03/04	2	655	2061

The rating is written once. The other table just points at it with the number 2.

This is the single most important idea today

Store every fact exactly once, in the table where it belongs, and point at it from everywhere else. Everything a database does well — staying consistent, being quick to change, refusing to accept nonsense — follows from that one habit.

OUR EXAMPLE

The boiler house database — five tables

boilers — 3 rows

boiler_id (key)
code, name
rating_tph, fuel

readings — 540 rows

reading_id (key)
boiler_id →
ts, steam_tph,
stack_temp_c, o2_pct

water_tests — 15 rows

test_id (key)
boiler_id →
test_date,
silica_ppm, ph

operators — 6 rows

operator_id (key)
full_name
shift

daily_logs — 90 rows

log_id (key)
boiler_id → operator_id →
log_date, steam_t, fuel_gj

Read the arrows out loud

- A row in readings does not say “Boiler 2”. It says boiler_id = 2 — and you look 2 up in the boilers table.
- A row in daily_logs points at two things: which boiler, and which operator was on.
- Everything else is just numbers and dates measured on the day.

01

PART ONE

Asking the database questions

SQL is how you ask. It is closer to English than to programming, and you only need a handful of words.

- SELECT — which columns
- FROM — which table
- WHERE — which rows
- ORDER BY and LIMIT — sorting and trimming

PART ONE

Every query has the same shape

```
SELECT   name, rating_tph
FROM     boilers
WHERE    rating_tph > 30
ORDER BY name
LIMIT    10;
```

SELECT	which columns you want back
FROM	which table to look in
WHERE	which rows to keep (optional)
ORDER BY	how to sort them (optional)
LIMIT	how many to show (optional)

Only the first two lines are compulsory. Add the others when you need them — and always in that order. If you can read those five lines, you can read most of the SQL you will ever meet.

Two small rules that catch everybody

Text goes in single quotes

`code = 'B-02'  is right.  code = "B-02"  is not.`

Numbers do not

`boiler_id = 2  ·  rating_tph > 30`

PART ONE

Choosing columns, then choosing rows

Everything, so you can look around

```
SELECT * FROM boilers;
```

boiler_id	code	name	rating_tph	fuel
1	B-01	Boiler 1	35.0	Natural gas
2	B-02	Boiler 2	35.0	Natural gas
3	B-03	Boiler 3	30.0	Natural gas

* means “every column”. Fine for looking; name your columns in anything you keep.

Just what you asked for

```
SELECT name, rating_tph
FROM   boilers
WHERE  rating_tph > 30;
```

name	rating_tph
Boiler 1	35.0
Boiler 2	35.0

SELECT picked the columns. WHERE picked the rows. That is all a simple query does.

Things you can put after WHERE

`rating_tph > 30`	greater than (also <, >=, <=)
`boiler_id = 2`	equal to
`code = 'B-02'`	equal to some text
`boiler_id = 2 AND o2_pct < 3`	both must be true
`ts LIKE '2026-04-01%'`	text that starts with something

PART ONE

Sorting, trimming, and counting

```
SELECT *
FROM   readings
ORDER  BY stack_temp_c DESC
LIMIT  5;
```

“Show me the five hottest readings.” DESC means biggest first; leave it out and you get smallest first.

```
SELECT COUNT(*) AS n_readings
FROM   readings;
```

COUNT(*) answers “how many rows?”. AS just gives the answer column a readable name.

Predict, then run

Before you run a counting query, guess the answer.

3 boilers × 6 readings a day × 30 days = 540.

If the query says 540, you know it did what you meant.

If it says 180, you have accidentally asked about one boiler — and you have caught the mistake in five seconds instead of in your report.

HANDS-ON · LAB 1

Looking at the data

1. **Show the whole boilers table**
   Then answer out loud: what does one row represent?

2. **Name and rating of boilers above 30 t/h**
   SELECT picks the columns, WHERE picks the rows.

3. **How many rows are in readings?**
   Predict the number before you run it.

4. **The five hottest stack readings**
   All columns, hottest first. Notice anything?

5. **Every reading for boiler 2 on 1 April**
   Two conditions joined with AND.

Hint · Text needs single quotes. Numbers do not. That is the mistake almost everyone makes first.

02

PART TWO

One number instead of five hundred

Nobody wants 540 readings. They want the average — and usually the average for each boiler.

- COUNT, AVG, MIN, MAX, SUM
- GROUP BY — one answer per group
- Reading the answer like an engineer

PART TWO

Squeezing many rows into one number

`COUNT(*)`
how many rows

`AVG(x)`
the average

`MIN(x)`
the smallest

`MAX(x)`
the biggest

`SUM(x)`
the total

`ROUND(x, 1)`
tidy it up for reading

```
SELECT ROUND(AVG(steam_tph), 1) AS mean_steam
FROM   readings;
```

mean_steam
27.0

540 rows in, one number out. But this average mixes all three boilers together — which is exactly the problem the next slide solves.

A warning worth having

You can add up tonnes of steam and GJ of gas. Those are amounts.

You cannot add up temperatures or percentages. Ask for their average instead.

SQL will happily do the wrong one and give you a confident-looking number.

PART TWO

GROUP BY: one answer per boiler

Add one line, and “the average” becomes “the average for each boiler”.

```
SELECT boiler_id,
       ROUND(AVG(stack_temp_c), 1) AS mean_stack_c
FROM   readings
GROUP  BY boiler_id;
```

boiler_id	mean_stack_c
1	124.9
2	183.1
3	128.2

Stop and look at that answer

Boiler 2's flue gas leaves nearly 60 °C hotter than the other two. You have just found something real, in three lines, by summarising 540 rows you never had to read.

How to read it	The database sorts the rows into piles — one pile per boiler_id — and works out the average within each pile.
The one rule	Every column in SELECT is either the thing you grouped by, or wrapped in AVG / COUNT / SUM. Nothing else is allowed.
Always add COUNT(*)	It tells you how many rows went into each average. 180 each here — if one boiler had 40, you would want to know why.

HANDS-ON · LAB 2

Summarising

1. **Average steam flow across all readings**
   One number from 540 rows.

2. **Average stack temperature for each boiler**
   Look hard at the answer. Is anything odd?

3. **Lowest and highest stack temperature per boiler**
   MIN and MAX, grouped.

4. **How many readings does each boiler have?**
   A boring answer is a good answer — it is a check.

5. **Total steam and total gas per boiler**
   From daily_logs this time, not readings.

Hint · Every one of these is the same query with a different function in the middle.

03

PART THREE

Putting the tables back together

We split the data up so every fact is stored once. A JOIN is how you bring it back together when you want to read it.

- Why readings only says “2”
- JOIN … ON — matching the rows up
- JOIN plus GROUP BY — the shape of every report

PART THREE

The readings table does not know what “2” means

`readings`

reading_id	boiler_id	ts	stack_temp_c
4	2	2026-04-01 00:00	177.6
7	2	2026-04-01 04:00	179.1

Useful, but nobody wants to read “2”. And the rating is not here at all.

`boilers`

boiler_id	code	name	rating_tph
1	B-01	Boiler 1	35.0
2	B-02	Boiler 2	35.0
3	B-03	Boiler 3	30.0

Everything about the boiler, written down once.

A JOIN is a lookup

“Take each reading, find the row in boilers whose boiler_id matches, and glue the two rows together.” That is all it is — the same thing you would do with your finger.

```
SELECT b.name, r.ts, r.stack_temp_c
FROM   readings r
JOIN   boilers  b  ON  b.boiler_id = r.boiler_id;
--                     ^^^^^^^^^^^^^^^^^^^^^^^^ where they match
```

PART THREE

Join to get the names, group to get the answer

```sql
SELECT b.code,
       ROUND(AVG(r.stack_temp_c), 1) AS mean_stack_c
FROM   readings r
JOIN   boilers  b  ON  b.boiler_id = r.boiler_id
GROUP  BY b.code;
```

code	mean_stack_c
B-01	124.9
B-02	183.1
B-03	128.2

The same answer as before — now readable by someone who has never seen the database.

This is the shape of almost every report you will ever write

Join to bring in the names and the reference data · group to summarise · sort to put the interesting row at the top.

`JOIN`	so the answer has names in it, not id numbers
`GROUP BY`	so you get one row per boiler, month, or shift
`ORDER BY`	so the worst one is at the top where people will see it

HANDS-ON · LAB 3

Joining, and answering the question

1. **Readings with the boiler's name**
   Join readings to boilers. First 10 rows.

2. **Daily logs with boiler code and operator name**
   Two joins. Same line, written twice.

3. **Average stack temperature per boiler code**
   Exercise 2 question 2, with a join on top.

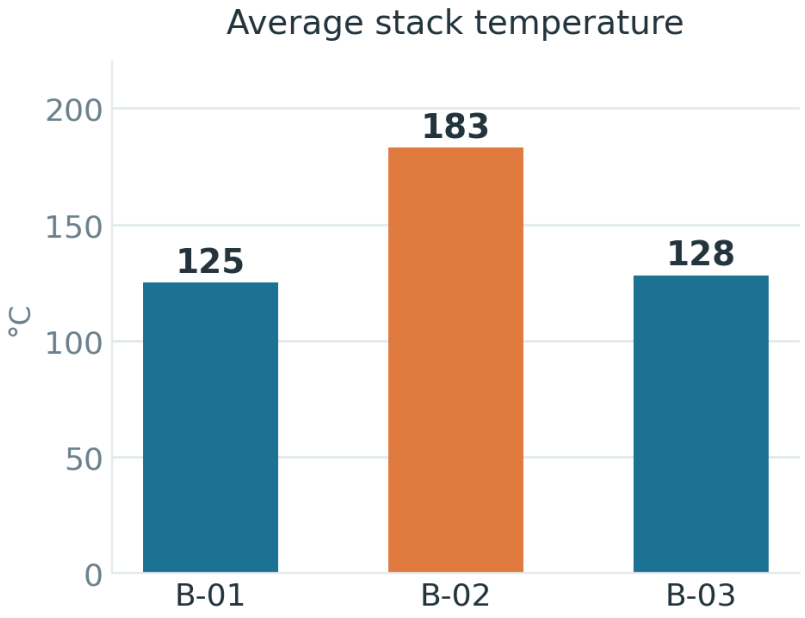
4. **Gas used per tonne of steam, per boiler**
   Total gas ÷ total steam. Which is worst?

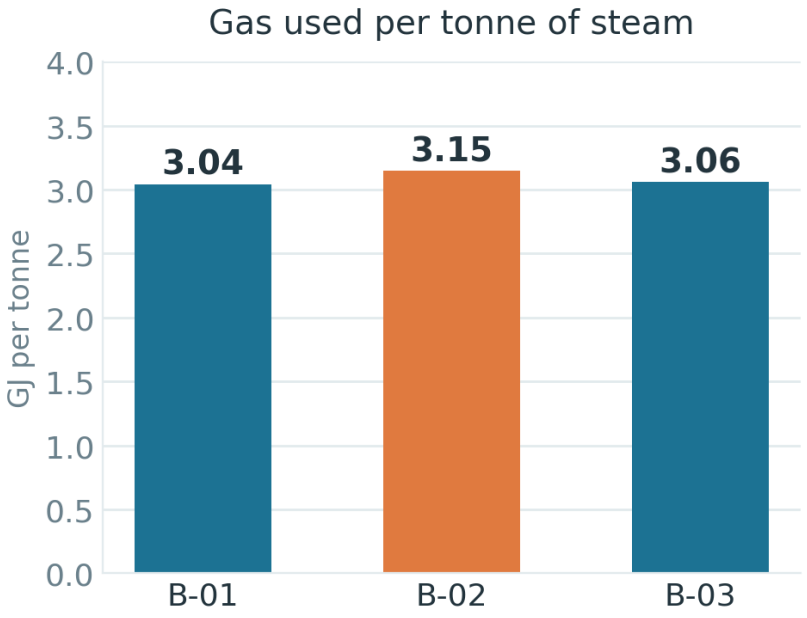
5. **No query — just think**
   Put questions 3 and 4 side by side. Why is B-02 using more gas?

Hint · Build the query one line at a time. Run it after every line you add.

THE ANSWER

What you found, in two pictures





B-02 runs 57 °C hotter

and burns 3 % more gas for every tonne of steam it makes.

That heat is going up the chimney.

Two queries. Two numbers. They agree.

The rule of thumb is about 1 % of boiler efficiency for every 20 °C of extra stack temperature. 57 °C predicts roughly 2.8 %. The fuel figures, worked out from completely different columns, say 3 %. When two independent numbers agree like that, you have something worth acting on — about 1,800 GJ of gas wasted in one month, near 16,000 USD.

TO TAKE AWAY

Four things, and none of them is syntax

1	**A table is a list of one kind of thing**	If you cannot say what one row is in four words, it should be two tables.
2	**Store every fact exactly once**	Everything else points at it. That is why a database stays correct and a spreadsheet drifts.
3	**GROUP BY is the one to remember**	Turning 540 rows into one number per boiler is what found the problem today.
4	**The computer counts, you interpret**	SQL said 183 °C and 3.15 GJ per tonne. Only you can say that means the tubes need cleaning.

You will meet the same five words again in every plant historian, maintenance system and laboratory database you ever touch.